

PROJECT TITLE IN CAPITAL LETTERS

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award
of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

STUDENT NAME – 25B11AI573

STUDENT NAME – 25B11AI261

STUDENT NAME – 25B11AIB75

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “ **University Academic Management System** ” is being submitted by **Manish Mandal** (Roll No. 25B11AI573), **DL Vaishnavi** (Roll No. 25B11AI261) and **T Meghana** (Roll No. 25B11AIB75) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)
professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D. Associate
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “ **University Academic Management System** ” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of **Mr. T.Srinivasulu** Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

Manish Mandal (25B11AI573)

D L Vaishnavi (25B11AI261)

T Meghana(25B11AIB75)

Date: 27-sep-2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide **Mr. T. Srinivasulu**, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **Dr. K NagaBhargavi**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

Manish Mandal	25B11AI573
DL Vaishnavi	25B11AI261
T Meghana	25B11AIB75

ABSTRACT

The **University Academic Management Database Application** is designed to automate and efficiently manage academic activities within a university through a centralized relational database system. The proposed application provides a structured and organized approach for storing, maintaining, and retrieving important academic information related to **students, departments, faculty, courses, enrollments, and examination results**. The system is developed to overcome common problems associated with manual record maintenance and scattered data, such as data duplication, inconsistency, difficulty in searching records, and challenges in generating academic reports.

The database provides a centralized platform where academic information can be stored systematically and accessed efficiently using **Structured Query Language (SQL)**. Primary keys and foreign keys are used to establish relationships between different tables and maintain **referential integrity**. Various constraints, including uniqueness and required fields, help ensure that the stored information remains accurate and consistent. The system also supports operations such as adding and managing student records, maintaining department and faculty information, managing courses, registering students for courses, and recording examination results.

The database design follows the principles of **normalization** and is structured up to **Third Normal Form (3NF)**. This helps minimize data redundancy, prevent insertion, deletion, and update anomalies, and improve overall data consistency. The major entities of the system include Department, Student, Faculty, Course, Enrollment, and Result, which are connected through appropriate relationships.

The proposed system also supports efficient SQL-based data retrieval, filtering, joining, sorting, aggregation, and academic reporting. It provides a strong database foundation for managing university academic operations in a systematic manner. Furthermore, the system can be extended in the future to include additional functionalities such as attendance management, fee management, timetable management, library management, examination scheduling, automated reporting, and web or mobile interfaces. Overall, the University Academic Management Database Application provides a reliable and structured solution for managing academic records while improving data organization, accessibility, integrity, and consistency.

LIST OF FIGURES

Figure 1. ER Diagram of University Academic Management Database

Figure 2. Proposed Three-Layer Database Application Architecture

Figure 3. Student Management Screen – Placeholder

Figure 4. Course Management Screen – Placeholder

Figure 5. Enrollment / Result Screen – Placeholder

Figure 6. Report Generation Screen – Placeholder



LIST OF TABLES

Table 1. Major Database Entities

Table 2. Department Table Structure

Table 3. Student Table Structure

Table 4. Faculty Table Structure

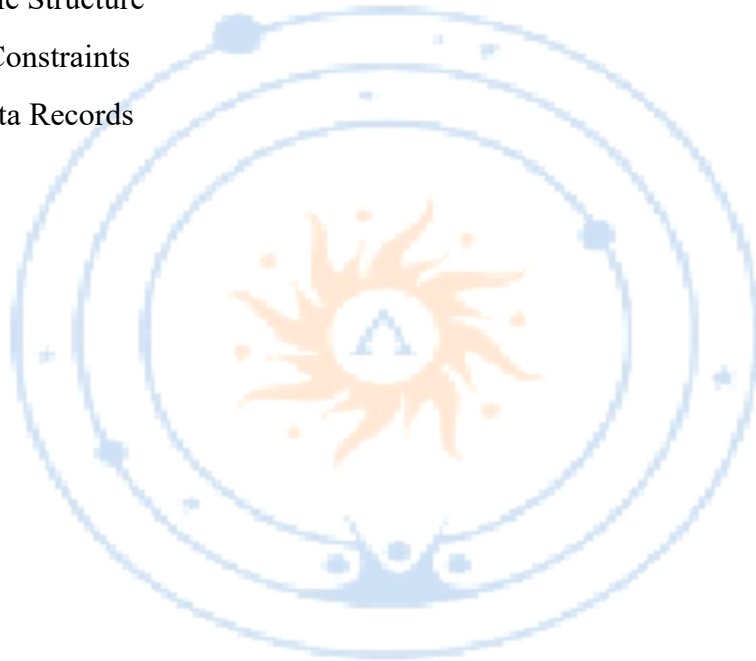
Table 5. Course Table Structure

Table 6. Enrollment Table Structure

Table 7. Result Table Structure

Table 8. Keys and Constraints

Table 9. Sample Data Records



1. INTRODUCTION

1.1 Brief Information about the Project

A University Academic Management System is a database application that manages academic information and administrative processes. The project provides a centralized relational database for departments, students, faculty, courses, enrollments, and examination results. It is intended to replace fragmented manual records and multiple spreadsheet files with a structured database that supports consistent storage and efficient retrieval.

1.2 Motivation and Contribution of Project

The project is motivated by common problems in academic record management such as duplicate records, time-consuming searches, inconsistent information, and difficulty in generating reports. A centralized database provides a systematic approach to storing related academic data and enforcing relationships through keys and constraints.

- Centralizing academic records in one relational database.
- Reducing duplicate and inconsistent information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster academic queries and report generation.
- Providing a foundation for student, faculty, course, enrollment, and result management.

1.3 Objectives of the Project

- Develop a centralized academic database.
- Manage student records.
- Maintain faculty information.
- Store department details.
- Handle course registration and student enrollments.
- Record examination results.
- Reduce duplicate data and update anomalies.
- Maintain referential integrity.
- Support efficient SQL queries and academic reports.

1.4 Scope of the Project

The scope covers database-oriented management of university academic information. The core scope includes department, student, faculty, course, enrollment, and result data. The database can be used as the backend for an administrative application and can be extended later with attendance, fee, timetable, library, examination scheduling, and web/mobile interfaces.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual record maintenance.
- Multiple Excel sheets or disconnected files.
- Duplicate records and inconsistent information.
- Time-consuming searches.
- Difficulty in generating consolidated reports.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes academic information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate. The system supports student, faculty, department, course, enrollment, and result management.

- Centralized database
- Student management
- Faculty management
- Course management
- Enrollment management
- Result processing
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Student Management	Add, update, search, and display student details.
Department Management	Store and maintain department information.
Faculty Management	Store faculty details and associate faculty with departments.
Course Management	Maintain courses, credits, departments, and faculty associations.
Enrollment Management	Register students for courses and maintain enrollment details.
Result Management	Store examination marks and grades associated with an enrollment.
Search and Reporting	Retrieve student, course, enrollment, and result information using SQL queries.

2.4 Non-Functional Requirements

- Performance – queries should return results efficiently for normal academic workloads.
- Integrity – invalid references and duplicate key values should be prevented.
- Security – database privileges should be controlled for administrative users.
- Usability – forms and reports should be understandable to users.
- Reliability – transactions should preserve consistent database state.
- Scalability – the design should support growth in students, courses, and academic records.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Department	Stores department information.
Student	Stores student records.
Faculty	Stores faculty details.
Course	Stores course information.
Enrollment	Represents student course registration.
Result	Stores examination result information linked to enrollment.

3.2 ER Model

The ER model represents the major entities and their relationships. The source documentation identifies Department–Student, Department–Faculty, Department–Course, Faculty–Course, Student–Enrollment, Course–Enrollment, and Enrollment–Result relationships. The diagram in the supplied project material also shows attributes such as Student_ID, Roll_No, Student_Name, Department_ID, Faculty_ID, Course_ID, Enrollment_ID, Result_ID, marks, grade, and related academic attributes.

3.3 Relationships

Relationship	Description
Department → Student	One department can have many students.
Department → Faculty	One department can have many faculty members.
Department → Course	One department can offer many courses.
Faculty → Course	Faculty members can be associated with courses.
Student → Enrollment	A student can have multiple enrollment records.
Course → Enrollment	A course can have multiple enrollment records.
Enrollment → Result	An enrollment can have an associated result record.

4. ER DIAGRAM

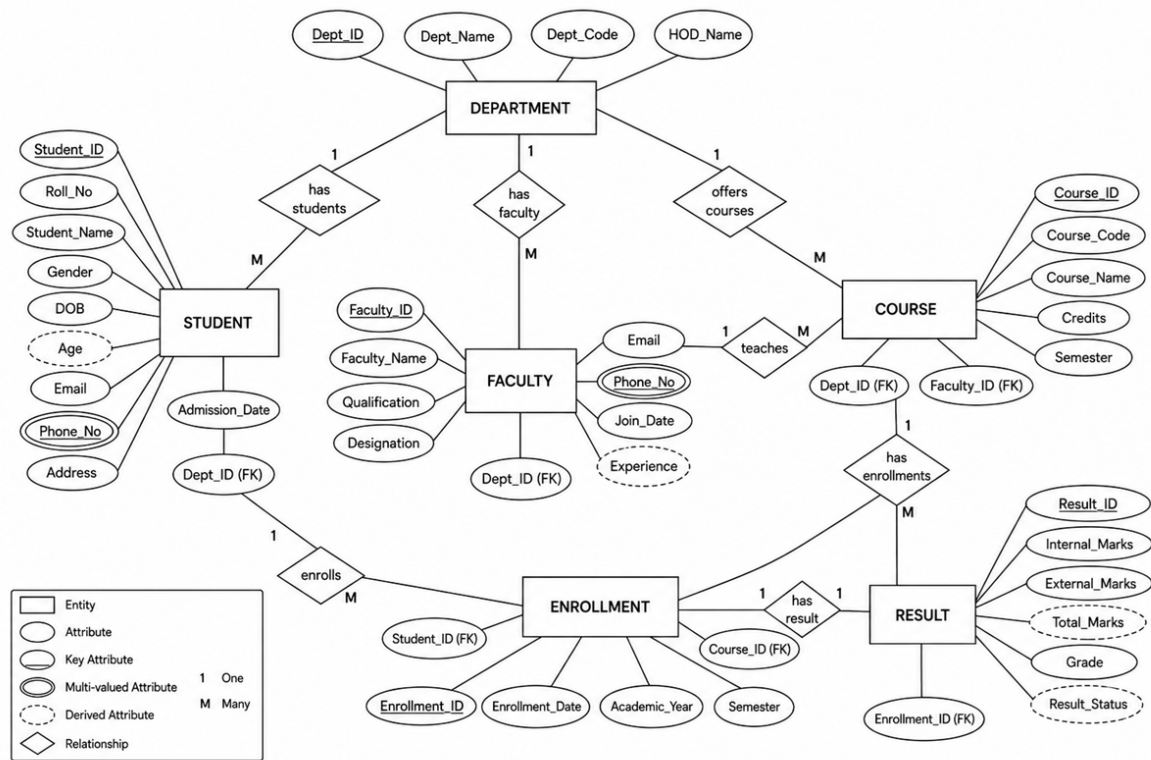


Figure 1: ER Diagram of University Academic Management Database

The ER diagram summarizes the entity structure and cardinalities described in the supplied DBMS project material. It is a documentation-level representation; the final project team should align attribute names and cardinalities with the actual implemented database.

5. RELATIONAL SCHEMA

Relation	Relational Schema
DEPARTMENT	Department_ID (PK), Department_Name, Department_Code, HOD_Name
STUDENT	Student_ID (PK), Student_Name, Roll_Number, Gender, DOB, Age, Email, Phone_No, Admission_Date, Address, Department_ID (FK)
FACULTY	Faculty_ID (PK), Faculty_Name, Qualification, Designation, Email, Phone_No, Join_Date, Experience, Department_ID (FK)
COURSE	Course_ID (PK), Course_Code, Course_Name, Credits, Semester, Department_ID (FK), Faculty_ID (FK)
ENROLLMENT	Enrollment_ID (PK), Student_ID (FK), Course_ID (FK), Enrollment_Date, Academic_Year, Semester
RESULT	Result_ID (PK), Enrollment_ID (FK), Internal_Marks, External_Marks, Total_Marks, Grade, Result_Status

The relational schema follows the structure presented in the supplied project material. The Student and Faculty relations reference Department; Course references Department and Faculty; Enrollment references Student and Course; and Result references Enrollment.

6. TABLE DESIGN

6.1 Department Table

Attribute	Data Type	Constraint	Description
Department_ID	INT / NUMBER	PK, NOT NULL	Unique department identifier
Department_Name	VARCHAR	NOT NULL	Department name
Department_Code	VARCHAR	UNIQUE	Department code
HOD_Name	VARCHAR	—	Head of Department name

6.2 Student Table

Attribute	Data Type	Constraint	Description
Student_ID	INT	PK	Unique student identifier
Student_Name	VARCHAR	NOT NULL	Student name
Roll_Number	VARCHAR	UNIQUE	Institution roll number
Gender	VARCHAR	—	Gender value as required by implementation
DOB	DATE	—	Date of birth
Age	INT	—	Age, if maintained
Email	VARCHAR	UNIQUE	Student email
Phone_No	VARCHAR	—	Contact number
Admission_Date	DATE	—	Admission date
Address	VARCHAR	—	Address
Department_ID	INT	FK	References Department

6.3 Faculty Table

Attribute	Data Type	Constraint	Description
Faculty_ID	INT	PK	Unique faculty identifier
Faculty_Name	VARCHAR	NOT NULL	Faculty name
Qualification	VARCHAR	—	Academic qualification
Designation	VARCHAR	—	Faculty designation
Email	VARCHAR	UNIQUE	Faculty email
Phone_No	VARCHAR	—	Contact number
Join_Date	DATE	—	Joining date
Experience	INT	—	Experience in years, if maintained
Department_ID	INT	FK	References Department

6.4 Course Table

Attribute	Data Type	Constraint	Description
Course_ID	INT	PK	Unique course identifier
Course_Code	VARCHAR	UNIQUE	Course code
Course_Name	VARCHAR	NOT NULL	Course name
Credits	INT	CHECK	Credit value
Semester	INT	—	Semester offered
Department_ID	INT	FK	References Department
Faculty_ID	INT	FK	References Faculty

6.5 Enrollment Table

Attribute	Data Type	Constraint	Description
Enrollment_ID	INT	PK	Unique enrollment identifier
Student_ID	INT	FK	References Student
Course_ID	INT	FK	References Course
Enrollment_Date	DATE	—	Enrollment date
Academic_Year	VARCHAR	—	Academic year
Semester	INT	—	Semester

6.6 Result Table

Attribute	Data Type	Constraint	Description
Result_ID	INT	PK	Unique result identifier
Enrollment_ID	INT	FK	References Enrollment
Internal Marks	DECIMAL	—	Internal assessment marks
External Marks	DECIMAL	—	External examination marks
Total Marks	DECIMAL	—	Total marks
Grade	VARCHAR	—	Grade obtained
Result_Status	VARCHAR	—	Pass/fail or equivalent status

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Department_ID, Student_ID, Faculty_ID, Course_ID, Enrollment_ID, Result_ID
Foreign Key	Student.Department_ID; Faculty.Department_ID; Course.Department_ID; Course.Faculty_ID; Enrollment.Student_ID; Enrollment.Course_ID; Result.Enrollment_ID
NOT NULL	Required attributes such as names and identifiers
UNIQUE	Roll number, email, course code, and department code where applicable
CHECK	Values such as course credits and marks can be restricted to valid ranges
DEFAULT	Can be used for fields such as result status or semester when appropriate

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The supplied project template explicitly requires documentation from UNF to 3NF. The following example uses an academic registration record to illustrate the process.

7.1 Unnormalized Form (UNF)

A single academic record might contain Student_ID, Student_Name, Department_Name, Course_ID, Course_Name, Faculty_Name, multiple course values, and multiple result values in one record. Repeating course groups and repeated department/faculty information create redundancy.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains atomic values. Each course enrollment is represented as a separate row. Student and department details may still be repeated across multiple enrollment records.

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on a composite key are removed. Student-specific attributes depend on Student_ID, course-specific attributes depend on Course_ID, and enrollment-specific attributes depend on Enrollment_ID. These dependencies motivate separate Student, Course, and Enrollment relations.

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Department details are stored in Department rather than repeated in Student, Faculty, and Course. Faculty details are stored in Faculty rather than repeated in Course. Result details are linked to Enrollment through Result. The resulting structure supports the 3NF objective stated in the supplied project material.

Relation	Reason for Separation
Department	Stores department-specific attributes once.
Student	Stores student-specific attributes and a Department_ID reference.
Faculty	Stores faculty-specific attributes and a Department_ID reference.
Course	Stores course-specific attributes and references its department/faculty.
Enrollment	Stores the student–course registration relationship.
Result	Stores marks/grade information for an enrollment.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL is a documentation-ready implementation aligned with the supplied relational schema. Data types may be adjusted to the specific DBMS used by the student team.

```
CREATE TABLE Department (  
    Department_ID NUMBER PRIMARY KEY,  
    Department_Name VARCHAR2(100) NOT NULL,  
    Department_Code VARCHAR2(20) UNIQUE,  
    HOD_Name VARCHAR2(100)  
);  
  
CREATE TABLE Student (  
    Student_ID NUMBER PRIMARY KEY,  
    Student_Name VARCHAR2(100) NOT NULL,  
    Roll_Number VARCHAR2(30) UNIQUE,  
    Gender VARCHAR2(20),  
    DOB DATE,  
    Age NUMBER,  
    Email VARCHAR2(100) UNIQUE,  
    Phone_No VARCHAR2(15),  
    Admission_Date DATE,  
    Address VARCHAR2(200),  
    Department_ID NUMBER,  
    CONSTRAINT fk_student_dept  
        FOREIGN KEY (Department_ID)  
        REFERENCES Department(Department_ID)  
);  
  
CREATE TABLE Faculty (  
    Faculty_ID NUMBER PRIMARY KEY,  
    Faculty_Name VARCHAR2(100) NOT NULL,  
    Qualification VARCHAR2(100),  
    Designation VARCHAR2(50),  
    Email VARCHAR2(100) UNIQUE,  
    Phone_No VARCHAR2(15),  
    Join_Date DATE,  
    Experience NUMBER,  
    Department_ID NUMBER,  
    CONSTRAINT fk_faculty_dept  
        FOREIGN KEY (Department_ID)  
        REFERENCES Department(Department_ID)  
);  
  
CREATE TABLE Course (  
    Course_ID NUMBER PRIMARY KEY,  
    Course_Code VARCHAR2(20) UNIQUE,  
    Course_Name VARCHAR2(100) NOT NULL,  
    Credits NUMBER CHECK (Credits > 0),  
    Semester NUMBER,  
    Department_ID NUMBER,
```

```

Faculty_ID NUMBER,
CONSTRAINT fk_course_dept
    FOREIGN KEY (Department_ID)
    REFERENCES Department(Department_ID),
CONSTRAINT fk_course_faculty
    FOREIGN KEY (Faculty_ID)
    REFERENCES Faculty(Faculty_ID)
);

CREATE TABLE Enrollment (
    Enrollment_ID NUMBER PRIMARY KEY,
    Student_ID NUMBER NOT NULL,
    Course_ID NUMBER NOT NULL,
    Enrollment_Date DATE,
    Academic_Year VARCHAR2(20),
    Semester NUMBER,
    CONSTRAINT fk_enroll_student
        FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    CONSTRAINT fk_enroll_course
        FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

CREATE TABLE Result (
    Result_ID NUMBER PRIMARY KEY,
    Enrollment_ID NUMBER NOT NULL,
    Internal_Marks NUMBER,
    External_Marks NUMBER,
    Total_Marks NUMBER,
    Grade VARCHAR2(5),
    Result_Status VARCHAR2(20),
    CONSTRAINT fk_result_enrollment
        FOREIGN KEY (Enrollment_ID)
        REFERENCES Enrollment(Enrollment_ID)
);

```

8.2 DML – Data Manipulation Language

```
INSERT INTO Department VALUES (1,'Information Technology','IT','Dr. Faculty Name');
```

```
INSERT INTO Student
(Student_ID, Student_Name, Roll_Number, Email, Department_ID)
VALUES
(101,'Student One','IT01','student1@example.edu',1);
```

```
INSERT INTO Faculty
(Faculty_ID, Faculty_Name, Designation, Department_ID)
VALUES
(201,'Faculty One','Assistant Professor',1);
```

```
INSERT INTO Course
(Course_ID, Course_Code, Course_Name, Credits, Semester, Department_ID, Faculty_ID)
VALUES
```

```
(301,'23IT05','Database Management Systems',4,5,1,201);
```

```
INSERT INTO Enrollment  
(Enrollment_ID, Student_ID, Course_ID, Enrollment_Date, Academic_Year, Semester)  
VALUES  
(401,101,301,SYSDATE,'2026-27',5);
```

```
INSERT INTO Result  
(Result_ID, Enrollment_ID, Internal_Marks, External_Marks, Total_Marks, Grade,  
Result_Status)  
VALUES  
(501,401,28,62,90,'A+', 'PASS');
```

8.3 DQL – Data Query Language

```
SELECT * FROM Student;
```

```
SELECT Student_ID, Student_Name, Email  
FROM Student  
WHERE Department_ID = 1;
```

```
SELECT Course_ID, Course_Code, Course_Name, Credits  
FROM Course  
ORDER BY Course_Name;
```

```
SELECT s.Student_Name, d.Department_Name  
FROM Student s  
JOIN Department d ON s.Department_ID = d.Department_ID;
```

```
SELECT s.Student_Name, c.Course_Name, e.Academic_Year, e.Semester  
FROM Student s  
JOIN Enrollment e ON s.Student_ID = e.Student_ID  
JOIN Course c ON e.Course_ID = c.Course_ID;
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display all departments

```
SELECT * FROM Department;
```

Output:

department_id	department_name	hod	office_no
10	CSE	Dr. Ramesh	A101
20	AIML	Dr. Suresh	A102
30	ECE	Dr. Kavitha	B101
40	EEE	Dr. Prasad	B102
50	MECH	Dr. Naresh	C101
60	CIVIL	Dr. Rao	C102
70	IT	Dr. Lakshmi	D101
80	MBA	Dr. Rekha	E101
90	MCA	Dr. Deepak	E102
100	BBA	Dr. Anita	F101

Q2. Display all students

```
SELECT * FROM Student;
```

Output:

student_id	name	gender	dob	phone	email	address	departm
101	Rahul	M	"2004-01-10T00:00:00.000Z"	9876543210	rahul@gmail.com	Kakinada	10
102	Sneha	F	"2004-02-15T00:00:00.000Z"	9876543211	sneha@gmail.com	Rajahmundry	20
103	Vamsi	M	"2004-03-20T00:00:00.000Z"	9876543212	vamsi@gmail.com	Amalapuram	30
104	Anusha	F	"2004-04-25T00:00:00.000Z"	9876543213	anusha@gmail.com	Vizag	40
105	Sai	M	"2004-05-18T00:00:00.000Z"	9876543214	sai@gmail.com	Kakinada	50
106	Pooja	F	"2004-06-22T00:00:00.000Z"	9876543215	pooja@gmail.com	Eluru	60
107	Rohit	M	"2004-07-11T00:00:00.000Z"	9876543216	rohit@gmail.com	Guntur	70
108	Keerthi	F	"2004-08-13T00:00:00.000Z"	9876543217	keerthi@gmail.com	Nellore	80
109	Harsha	M	"2004-09-19T00:00:00.000Z"	9876543218	harsha@gmail.com	Vijayawada	90
110	Divya	F	"2004-10-30T00:00:00.000Z"	9876543219	divya@gmail.com	Hyderabad	100

Q3. Display all faculty

```
SELECT * FROM Faculty;
```

Output:

faculty_id	faculty_name	qualification	phone	email	department_id
201	Rajesh	PhD	9000000001	rajesh@gmail.com	10
202	Srinivas	PhD	9000000002	srinivas@gmail.com	20
203	Priya	M.Tech	9000000003	priya@gmail.com	30
204	Anil	PhD	9000000004	anil@gmail.com	40
205	Meena	M.Tech	9000000005	meena@gmail.com	50
206	Ravi	PhD	9000000006	ravi@gmail.com	60
207	Deepa	PhD	9000000007	deepa@gmail.com	70
208	Kiran	MBA	9000000008	kiran@gmail.com	80
209	Divya	MCA	9000000009	divya@gmail.com	90
210	Ajay	MBA	9000000010	ajay@gmail.com	100

Q4. Display all courses

```
SELECT * FROM Course;
```

Output:

course_id	course_name	credits	semester	faculty_id	department_id
301	DBMS	4	3	201	10
302	Machine Learning	4	5	202	20
303	Digital Electronics	3	3	203	30
304	Power Systems	4	4	204	40
305	Thermodynamics	4	4	205	50
306	Surveying	3	3	206	60
307	Python	3	2	207	70
308	Marketing	4	2	208	80
309	Java	4	2	209	90
310	Business Analytics	4	3	210	100

Q5. Students from a department

```
SELECT * FROM Student WHERE Department_ID = 1;
```

Output:

[SCREENSHOT PLACEHOLDER]

Q6. Courses with more than 3 credits

```
SELECT * FROM Course WHERE Credits > 3;
```

Output:

course_id	course_name	credits	semester	faculty_id	department_id
301	DBMS	4	3	201	10
302	Machine Learning	4	5	202	20
304	Power Systems	4	4	204	40
305	Thermodynamics	4	4	205	50
308	Marketing	4	2	208	80
309	Java	4	2	209	90
310	Business Analytics	4	3	210	100

Q7. Students sorted by name

```
SELECT * FROM Student ORDER BY Student_Name;
```

Output:

student_id	name	gender	dob	phone	email	address	departm
104	Anusha	F	"2004-04-25T00:00:00.000Z"	9876543213	anusha@gmail.com	Vizag	40
110	Divya	F	"2004-10-30T00:00:00.000Z"	9876543219	divya@gmail.com	Hyderabad	100
109	Harsha	M	"2004-09-19T00:00:00.000Z"	9876543218	harsha@gmail.com	Vijayawada	90
108	Keerthi	F	"2004-08-13T00:00:00.000Z"	9876543217	keerthi@gmail.com	Nellore	80
106	Pooja	F	"2004-06-22T00:00:00.000Z"	9876543215	pooja@gmail.com	Eluru	60
101	Rahul	M	"2004-01-10T00:00:00.000Z"	9876543210	rahul@gmail.com	Kakinada	10
107	Rohit	M	"2004-07-11T00:00:00.000Z"	9876543216	rohit@gmail.com	Guntur	70
105	Sai	M	"2004-05-18T00:00:00.000Z"	9876543214	sai@gmail.com	Kakinada	50
102	Sneha	F	"2004-02-15T00:00:00.000Z"	9876543211	sneha@gmail.com	Rajahmundry	20
103	Vamsi	M	"2004-03-20T00:00:00.000Z"	9876543212	vamsi@gmail.com	Amalapuram	30

Q8. Courses sorted by credits

```
SELECT * FROM Course ORDER BY Credits DESC;
```

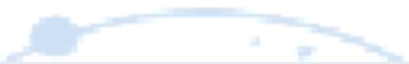
Output:

course_id	course_name	credits	semester	faculty_id	department_id
301	DBMS	4	3	201	10
302	Machine Learning	4	5	202	20
308	Marketing	4	2	208	80
309	Java	4	2	209	90
310	Business Analytics	4	3	210	100
304	Power Systems	4	4	204	40
305	Thermodynamics	4	4	205	50
303	Digital Electronics	3	3	203	30
306	Surveying	3	3	206	60
307	Python	3	2	207	70

Q9. Students with email addresses

```
SELECT Student_ID, Student_Name, Email FROM Student WHERE Email IS NOT NULL;
```

Output:



student_id	name	email
101	Rahul	rahul@gmail.com
102	Sneha	sneha@gmail.com
103	Vamsi	vamsi@gmail.com
104	Anusha	anusha@gmail.com
105	Sai	sai@gmail.com
106	Pooja	pooja@gmail.com
107	Rohit	rohit@gmail.com
108	Keerthi	keerthi@gmail.com
109	Harsha	harsha@gmail.com
110	Divya	divya@gmail.com

Q10. Department and students

```
SELECT d.Department_Name, s.Student_Name FROM Department d JOIN Student s ON  
d.Department_ID=s.Department_ID;
```

Output:

department_name	name
CSE	Rahul
AIML	Sneha
ECE	Vamsi
EEE	Anusha
MECH	Sai
CIVIL	Pooja
IT	Rohit
MBA	Keerthi
MCA	Harsha
BBA	Divya

Q11. Faculty with department

```
SELECT f.Faculty_Name, d.Department_Name FROM Faculty f JOIN Department d ON  
f.Department_ID=d.Department_ID;
```

Output:

faculty_name	department_name
Rajesh	CSE
Srinivas	AIML
Priya	ECE
Anil	EEE
Meena	MECH
Ravi	CIVIL
Deepa	IT
Kiran	MBA
Divya	MCA
Ajay	BBA

Q12. Courses with department

```
SELECT c.Course_Name, d.Department_Name FROM Course c JOIN Department d ON  
c.Department_ID=d.Department_ID;
```

Output:



course_name	department_name
DBMS	CSE
Machine Learning	AIML
Digital Electronics	ECE
Power Systems	EEE
Thermodynamics	MECH
Surveying	CIVIL
Python	IT
Marketing	MBA
Java	MCA
Business Analytics	BBA

Q13. Courses with faculty

```
SELECT c.Course_Name, f.Faculty_Name FROM Course c JOIN Faculty f ON  
c.Faculty_ID=f.Faculty_ID;
```

Output:



course_name	faculty_name
DBMS	Rajesh
Machine Learning	Srinivas
Digital Electronics	Priya
Power Systems	Anil
Thermodynamics	Meena
Surveying	Ravi
Python	Deepa
Marketing	Kiran
Java	Divya
Business Analytics	Ajay

Q14. Enrollment details

```
SELECT e.Enrollment_ID, s.Student_Name, c.Course_Name FROM Enrollment e JOIN Student s  
ON e.Student_ID=s.Student_ID JOIN Course c ON e.Course_ID=c.Course_ID;
```

Output:

enrollment_id	name	course_name
401	Rahul	DBMS
402	Sneha	Machine Learning
403	Vamsi	Digital Electronics
404	Anusha	Power Systems
405	Sai	Thermodynamics
406	Pooja	Surveying
407	Rohit	Python
408	Keerthi	Marketing
409	Harsha	Java
410	Divya	Business Analytics

Q15. Student result details

```
SELECT s.Student_Name, c.Course_Name, r.Total_Marks, r.Grade FROM Result r JOIN  
Enrollment e ON r.Enrollment_ID=e.Enrollment_ID JOIN Student s ON  
e.Student_ID=s.Student_ID JOIN Course c ON e.Course_ID=c.Course_ID;
```

OUTPUT:

student_name	course_name	marks	grade
Rahul	DBMS	89	A
Sneha	Machine Learning	92	A
Vamsi	Digital Electronics	78	B
Anusha	Power Systems	67	C
Sai	Thermodynamics	81	A
Pooja	Surveying	75	B
Rohit	Python	95	A
Keerthi	Marketing	72	B
Harsha	Java	69	C
Divya	Business Analytics	88	A

Q16. Students per department

```
SELECT Department_ID, COUNT(*) AS Student_Count FROM Student GROUP BY Department_ID;
```

Output:



department_id	student_count
80	1
70	1
60	1
40	1
30	1
90	1
10	1
50	1
100	1
20	1

Q17. Courses per department

```
SELECT Department_ID, COUNT(*) AS Course_Count FROM Course GROUP BY Department_ID;
```

Output:

department_id	course_count
80	1
70	1
60	1
40	1
30	1
90	1
10	1
50	1
100	1
20	1

Q18. Average course credits

```
SELECT AVG(Credits) AS Avg_Credits FROM Course;
```

Output:

avg_credits
3.7000000000000000

Q19. Maximum total marks

```
SELECT MAX(Total_Marks) AS Max_Marks FROM Result;
```

Output:

max_marks
95

Q20. Minimum total marks

```
SELECT MIN(Total_Marks) AS Min_Marks FROM Result;
```

Output:

min_marks
67

+ Code + Markdown

Q21. Average total marks

```
SELECT AVG(Total_Marks) AS Avg_Marks FROM Result;
```

Output:

avg_marks
80.6000000000000000

+ Code + Markdown

Q22. Count passed results

```
SELECT COUNT(*) AS Passed FROM Result WHERE Result_Status='PASS';
```

Output:

passed
10

Q23. Count failed results

```
SELECT COUNT(*) AS Failed FROM Result WHERE Result_Status='FAIL';
```

Output:

failed
0

[+ Code](#) [+ Markdown](#)

Q24. Students enrolled in DBMS

```
SELECT s.Student_Name FROM Student s JOIN Enrollment e ON s.Student_ID=e.Student_ID  
JOIN Course c ON e.Course_ID=c.Course_ID WHERE c.Course_Name='Database Management  
Systems';
```

Output:

student_name
Rahul

[+ Code](#) [+ Markdown](#)

Q25. Faculty teaching IT courses

```
SELECT DISTINCT f.Faculty_Name FROM Faculty f JOIN Course c ON  
f.Faculty_ID=c.Faculty_ID JOIN Department d ON c.Department_ID=d.Department_ID WHERE  
d.Department_Code='IT';
```

Output:

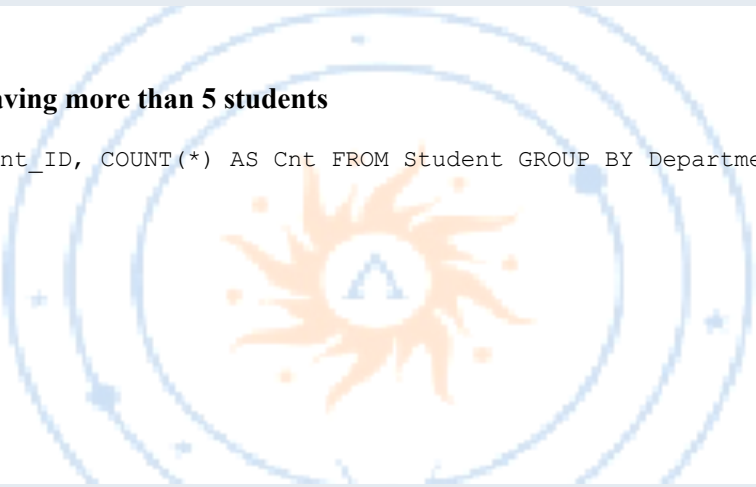
faculty_name
Deepa

+ Code + Markdown

Q26. Departments having more than 5 students

```
SELECT Department_ID, COUNT(*) AS Cnt FROM Student GROUP BY Department_ID HAVING  
COUNT(*) > 5;
```

Output:



```
SELECT Department_ID, COUNT(*) AS Cnt  
FROM Student  
GROUP BY Department_ID  
HAVING COUNT(*) >= 5;
```

+ Code + Markdown

A online PostgreSQL playground

Q27. Courses with 4 credits

```
SELECT Course_Name FROM Course WHERE Credits=4;
```

Output:

course_name	credits
DBMS	4
Machine Learning	4
Power Systems	4
Thermodynamics	4
Marketing	4
Java	4
Business Analytics	4

+ Code + Markdown

Q28. Students whose names start with A

```
SELECT * FROM Student WHERE Student_Name LIKE 'A%';
```

Output:

student_id	name	gender	dob	phone	email	address	department_ic
104	Anusha	F	"2004-04-25T00:00:00.000Z"	9876543213	anusha@gmail.com	Vizag	40

+ Code + Markdown

Q29. Search course by keyword

```
SELECT * FROM Course WHERE UPPER(Course_Name) LIKE '%DATABASE%';
```

Output:

course_id	course_name	credits	semester	faculty_id	department_id
301	DBMS	4	3	201	10

+ Code + Markdown

Q30. Students without department

```
SELECT * FROM Student WHERE Department_ID IS NULL;
```

Output:

⋮ ▶ 🗑

```
SELECT * FROM Student WHERE Department_ID IS NULL;
```

Postgres

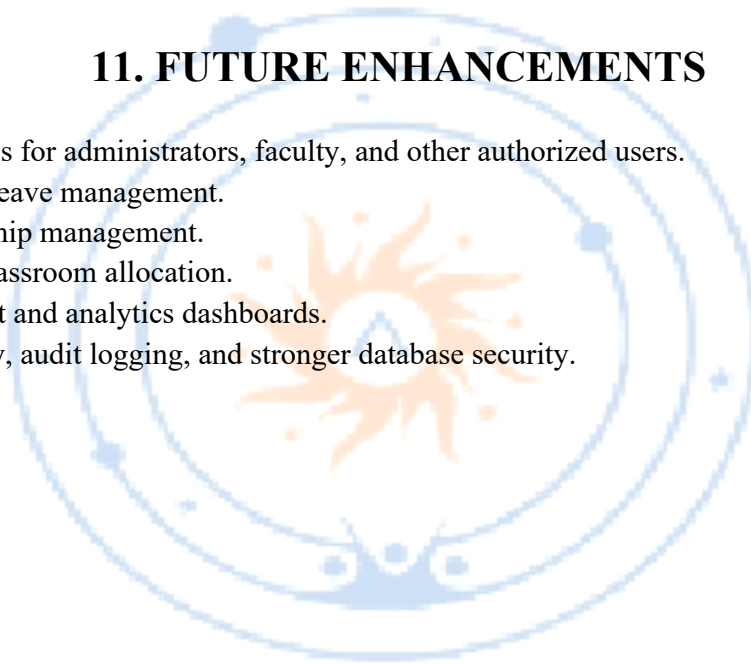
+ Code + Markdown

10. CONCLUSION

The University Academic Management Database Application provides a structured relational database solution for managing core academic information. The design organizes departments, students, faculty, courses, enrollments, and results into related relations and applies primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints to support data integrity. The normalization process up to 3NF reduces redundancy and update anomalies. The SQL implementation and query set demonstrate how the database can support insertion, modification, retrieval, aggregation, and academic reporting. The project therefore provides a suitable DBMS cornerstone implementation that can be extended into a complete academic information system.

11. FUTURE ENHANCEMENTS

- Role-based access for administrators, faculty, and other authorized users.
- Attendance and leave management.
- Fee and scholarship management.
- Timetable and classroom allocation.
- Automated report and analytics dashboards.
- Backup, recovery, audit logging, and stronger database security.



GITHUB REPOSITORY AND TEAM CONTRIBUTION

University Academic Management System

GitHub Repository

Repository Link: <https://github.com/25b11ai573/DBMS-Review-.git>

The project repository contains the files and documentation related to the University Academic Management System developed as part of the DBMS capstone project.

Team Contribution

The project was completed through collaborative contribution by all three team members. The work covered database design, SQL implementation, query preparation, testing, documentation, and project review.

S. No.	Team Member	Contribution
1	MANISH MANDAL 25B11AI573	Overall project development, database design, SQL implementation, query preparation, testing, and documentation.
2	D VAISHNAVI 25B11AI261	Database project work, SQL/query preparation, documentation, testing, and review of the project report.
3	T MEGHANA 25B11AIB75	Database project work, ER/database documentation, query preparation, testing, and review of the project report.

Major Areas Covered

- Database entities, relationships, relational schema, and table design.
- Normalization up to Third Normal Form (3NF).
- DDL, DML, DQL, joins, filtering, aggregation, and reporting queries.
- SQL outputs/screenshots, testing, validation, and project documentation.
- GitHub repository organization and project material submission.